

# InnoDB 聚簇索引、二级索引、回表、覆盖索引

## 1. 聚簇索引 (Clustered Index)

InnoDB 的数据是“按照主键顺序存储”的。

也就是说：

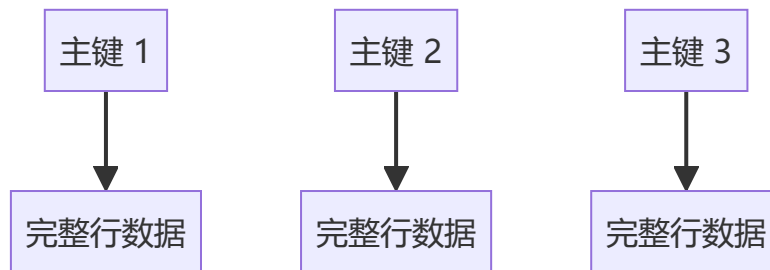
- 主键索引的叶子节点
- 存放的不是“指针”
- 而是“整行数据”

因此：

InnoDB 的主键索引本身就是数据文件

这就是聚簇索引。

### 聚簇索引结构



例如：

| id | name | age |
|----|------|-----|
| 1  | Tom  | 20  |
| 2  | Jack | 21  |

实际存储类似：

```
1 -> (1, Tom, 20)
2 -> (2, Jack, 21)
```

## 2. 二级索引 (Secondary Index)

除了主键索引以外的索引，都叫二级索引。

比如：

```
create index idx_name on user(name);
```

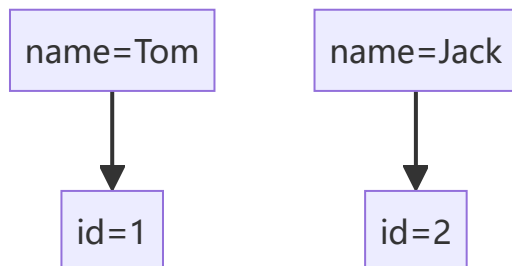
此时：

```
name -> 主键id
```

注意：

二级索引叶子节点存的不是整行数据，而是主键值。

## 二级索引结构



## 3. 回表（非常高频面试点）

假设 SQL：

```
select * from user where name = 'Tom';
```

执行过程：

### 第一步

先走二级索引：

```
name=Tom -> id=1
```

### 第二步

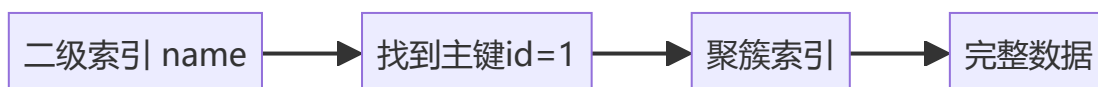
再根据主键 id=1

去聚簇索引查完整数据。

这个动作：

就叫回表

## 回表流程图



## 4. 覆盖索引（优化重点）

---

如果查询的数据：

在索引里已经全部存在

那么就不需要回表。

例如：

```
select id from user where name='Tom';
```

因为：

二级索引本来就存：

```
name -> id
```

所以：

不需要再查聚簇索引。

这就是：

覆盖索引

---

## 覆盖索引的核心价值

减少：

- 一次 B+ 树查询
- 随机 IO
- 回表成本

因此性能会明显提升。

---

## B+ 树适合索引的原因

数据库索引要求：

- 查询快
- 范围查询快
- 磁盘 IO 少
- 排序能力强

B+ 树正好满足。

---

# B+ 树核心特点

## 1. 多叉树，高度低

B+ 树一个节点可以存很多 key。

例如：

- 一个节点 16KB
- 一个 key 16B

则一个节点能存上千个 key。

因此树高很低：

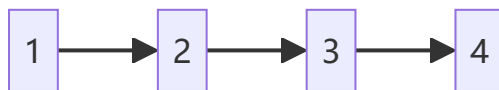
千万数据  
可能只需要3~4层

所以：

磁盘 IO 次数非常少。

## 2. 叶子节点形成链表

B+ 树叶子节点天然有序：



因此：

范围查询极强：

```
where id between 100 and 200
```

可以顺序扫描。

## 3. 非叶子节点不存数据

B+ 树：

- 非叶子节点只存 key
- 不存完整数据

所以：

一个节点能放更多 key。

树更矮。

IO 更少。

# 为什么用 B+ 树不用 B 树

## B 树的问题

B 树：

- 每个节点都存数据
- 导致节点能存的 key 变少
- 树更高

## 对比

| 对比项   | B树  | B+树  |
|-------|-----|------|
| 非叶子节点 | 存数据 | 不存数据 |
| 树高度   | 更高  | 更低   |
| 范围查询  | 较差  | 很强   |
| 顺序扫描  | 不方便 | 天然链表 |
| 磁盘IO  | 更多  | 更少   |

## 为什么数据库特别喜欢 B+ 树

数据库是：

“磁盘 IO 型系统”

不是 CPU 型系统。

真正慢的是：

磁盘随机读取

B+ 树的核心目标：

减少 IO 次数

## 事务隔离级别、MVCC 作用

SQL 标准四种隔离级别：

| 隔离级别             | 脏读 | 不可重复读 | 幻读   |
|------------------|----|-------|------|
| Read Uncommitted | 有  | 有     | 有    |
| Read Committed   | 无  | 有     | 有    |
| Repeatable Read  | 无  | 无     | 基本解决 |

| 隔离级别         | 脏读 | 不可重复读 | 幻读 |
|--------------|----|-------|----|
| Serializable | 无  | 无     | 无  |

MySQL InnoDB 默认：

Repeatable Read

# MVCC 的作用

MVCC：

Multi-Version Concurrency Control  
多版本并发控制

核心思想：

不加锁也能读

提高并发性能。

# MVCC 如何实现

InnoDB 每行数据：

隐藏字段：

trx\_id      事务id  
roll\_pointer 回滚指针

通过 undo log：

形成历史版本链。

# 版本链示意



读取时：

根据 ReadView 判断：

当前事务能看到哪个版本。

# 脏读 / 不可重复读 / 幻读 区别

# 1. 脏读（读到未提交数据）

---

事务 A:

```
update user set money=1000 where id=1;
```

但没提交。

事务 B:

读到了 1000。

结果:

事务 A 回滚了。

那么:

事务 B 读到了不存在的数据。

这就是脏读。

---

# 2. 不可重复读

---

同一个事务:

第一次读:

```
100
```

第二次读:

```
200
```

中间别人修改并提交了。

导致:

同一行数据前后读取不一致

---

# 3. 幻读

---

第一次:

```
select * from user where age > 20;
```

查到 10 条。

别人插入一条:

```
age=25
```

第二次再查:

变成 11 条。

像“幻觉一样多了一行”。

这就是幻读。

## 三者核心区别

| 问题    | 本质        |
|-------|-----------|
| 脏读    | 读到未提交数据   |
| 不可重复读 | 同一行被修改    |
| 幻读    | 查询结果集行数变化 |

## 索引失效常见场景：模糊前缀、函数运算、隐式转换

### 1. 模糊查询导致失效

#### 错误写法

```
where name like '%Tom'
```

因为：

不知道前缀是什么。

B+ 树无法定位起点。

只能全表扫描。

#### 正确写法

```
where name like 'Tom%'
```

可以利用索引。

### 2. 函数运算导致失效

#### 错误

```
where substr(name,1,3)='Tom'
```

索引保存的是原始值。



不是函数结果。

因此失效。

---

## 正确

```
where name like 'Tom%'
```

---

## 3. 隐式类型转换

例如：

```
name varchar
```

SQL：

```
where name = 123
```

MySQL 可能会：

```
cast(name as int)
```

导致索引失效。

---

## 行锁、间隙锁、临键锁基础理解

---

### 1. 行锁（Record Lock）

锁住：

某一行数据

例如：

```
update user set age=20 where id=1;
```

会锁：

```
id=1
```

---

### 2. 间隙锁（Gap Lock）

锁的不是数据。

而是：

两条记录之间的“空隙”

目的：

防止别人插入。

## 示例

当前数据：

10 20 30

事务：

```
select * from user where id between 10 and 20 for update;
```

可能锁：

(10,20)

此时：

别人不能插入：

15

## 3. 临键锁（Next-Key Lock）

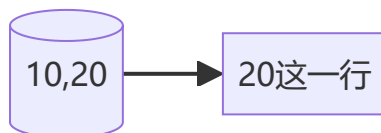
MySQL 默认 RR 隔离级别：

使用临键锁。

它是：

行锁 + 间隙锁

## 临键锁示意



既锁：

- 记录本身
- 前面的间隙

# 为什么需要间隙锁

核心目标：

解决幻读

否则：

事务第一次查询：

```
age > 20
```

第二次：

别人插入新数据。

结果集变化。

## 面试高频总结（建议背诵）

### InnoDB

- 主键索引是聚簇索引
- 二级索引叶子节点存主键
- 查询可能发生回表
- 覆盖索引避免回表

### B+ 树

- 多叉树
- 高度低
- IO 少
- 范围查询强
- 叶子节点链表

### MVCC

- 通过 undo log 形成版本链
- ReadView 决定可见性
- 提高并发
- 减少锁竞争

### 锁

- 行锁：锁单行
- 间隙锁：锁区间

- 临键锁：行锁+间隙锁
- RR 通过临键锁解决幻读